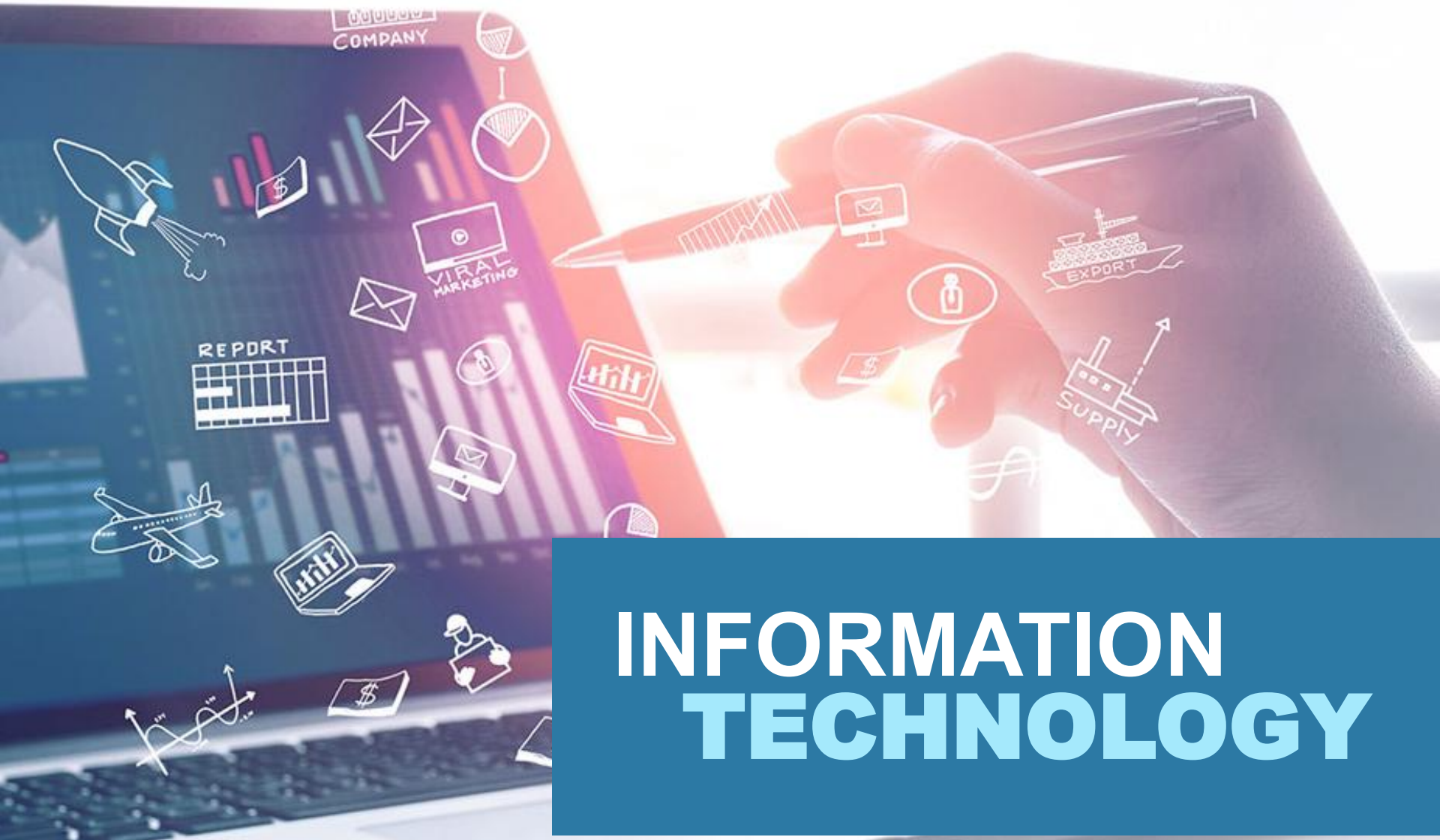
Data Structures and Algorithms



INFORMATION TECHNOLOGY

Data Structures and Algorithms

Objectives

- In this session, you will learn to:
  - Store and search data by using hashing
  - Identify the features of linked lists
  - Implement a singly-linked list

Implementing Hashing

- Binary search algorithm has some disadvantages:
  - It works only on sorted lists.
  - It requires a way to directly access the middle element of the list.
- An alternate searching algorithm that overcomes these limitations and provides good efficiency is hashing.

Defining Hashing

- Suppose, you have to search for the element corresponding to a given key value in a given list of elements.
- To retrieve the desired element, you have to search sequentially through the elements until the element with the desired key value is found.
- This method is very time consuming, especially if the list is very large.
- An effective solution to search the element would be to search it with the help of an offset address.
- You can calculate the offset address of the element by using a technique called hashing.

Defining Hashing (Contd.)

- The fundamental principle of hashing is to convert a given key value to an offset address to retrieve an element.
- Conversion of a key to an address is done by a relation (formula), which is known as a hashing function.
- The process of searching an element by using hashing can be summarized as:
  - Given a key, the hash function converts it into a hash value (location) within the range `1` to `n`, where `n` is the size of the storage (address) space that has been allocated for the records.
  - The element is then retrieved at the location generated.

Limitations of Hashing

- Two limitations of hashing are:
  - It may result in a collision.
  - It does not allow sequential access.

Resolving Collision

- A situation in which an attempt is made to store two keys at the same position is known as collision.
- Two records cannot occupy the same position. Therefore, a collision situation needs to be taken care of.
- Some methods of resolving collision are chaining and open addressing.
- The chaining method uses:
  - Links/pointers to resolve hash clashes.
  - Coalesced chaining and separate chaining techniques to resolve a collision.
- In the open addressing method:
  - Elements that produce a collision are stored at an alternate position in the hash table.
  - An alternate location is obtained by searching the hash table. This process is called probing.

Data Structures and Algorithms

Resolving Collision (Contd.)

- The following probing sequence can be used to search an empty position in the hash table:
  - Linear probing
  - Quadratic probing
  - Double hashing

Just a Minute

- In which one of the following hashing techniques, do you apply a second hash function if there is a collision?
  - Double hashing
  - Quadratic probing
  - Linear probing
  - Separate chaining

Just a Minute (Contd.)

- Solution:
  - Double hashing

Determining the Efficiency of Hashing

- Searching becomes faster using hashing as compared to any other searching method.
- The efficiency of hashing is ideally `O(1)`.
- However, because of any collision, the efficiency of hashing gets reduced.
- The efficiency of hashing in this case depends on the quality of the hash function.

Introduction to Linked Lists

- Suppose, you have to write an algorithm to generate and store all prime numbers between 1 and 10,00,000 and display them.
- Which data structure will you use to store the prime numbers?

Introduction to Linked Lists (Contd.)

- Consider the following algorithm, which uses an array to solve this problem:

```
1. Set I = 0.
2. Repeat step 3 varying n from 2 to 1000000.
3. If n is a prime number, then:
    a. Set A[I] = n. // If n is prime, store it
                     //in array
    b. I = I + 1.
4. Repeat step 5 varying J from 0 to I-1.
5. Display A[J].    // Display the prime numbers
                    // stored in the array
```

Introduction to Linked Lists (Contd.)

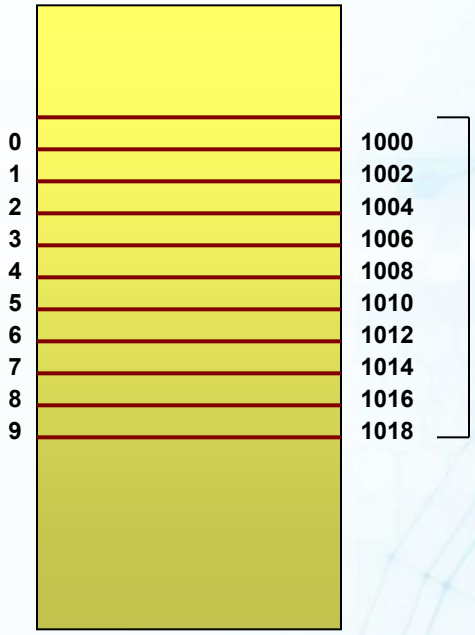
- What is the problem in this algorithm?
  - The number of prime numbers between 1 and 10,00,000 is not known. Since, you are using an array to store the prime numbers, you need to declare an array of arbitrarily large size to store the prime numbers.
  - Suppose you declare an array of size, `n`:
    - If the number of prime numbers between 1 and 10,00,000 is more than `n`, then all the prime numbers cannot be stored.
    - If the number of prime numbers is much less than `n`, a lot of memory space is wasted.

Introduction to Linked Lists (Contd.)

- Thus, you cannot use an array to store a set of elements, if you do not know the total number of elements in advance.
- How do you solve this problem?
  - By having some way in which you can allocate memory as and when it is required.

Dynamic Memory Allocation

- When you declare an array, a contiguous block of memory is allocated.
- Let us suppose, you declare an array of size 10 to store the first 10 prime numbers, as shown in the following figure.



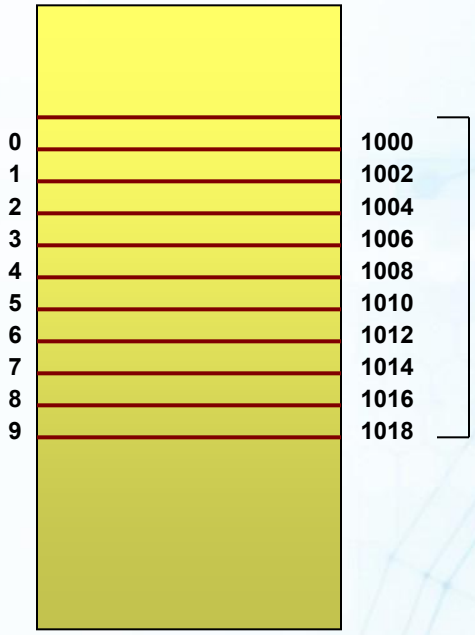
One contiguous block of memory allocated for the array to store 10 elements

Memory representation

Data Structures and Algorithms

Dynamic Memory Allocation (Contd.)

- If you know the address of the first element in the array, you can calculate the address of any other element as shown:
  - `Address of the first element + (size of the element × index of the element)`



One contiguous block of memory allocated for the array to store 10 elements

Memory representation

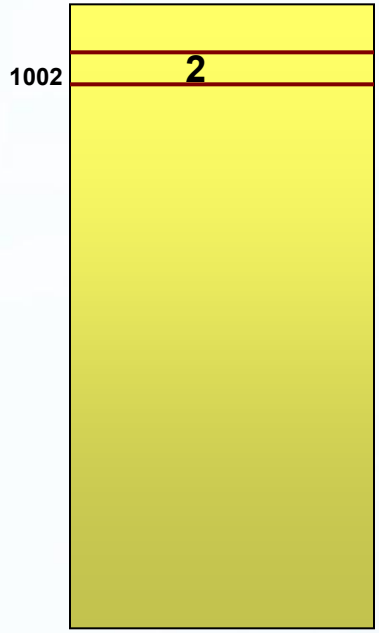
Dynamic Memory Allocation (Contd.)

- When memory is allocated dynamically, a block of memory is assigned arbitrarily from any location in the memory.
- Therefore, unlike arrays, these blocks may not be contiguous and may be spread randomly in the memory.

Data Structures and Algorithms

Dynamic Memory Allocation (Contd.)

- Let us see how this happens by allocating memory dynamically for the prime numbers.



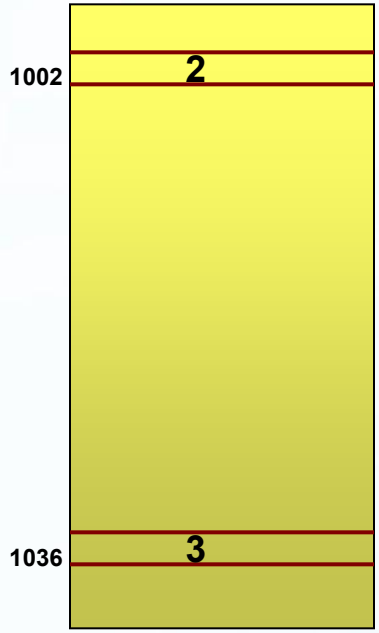
Memory allocated for 2

Memory representation

Data Structures and Algorithms

Dynamic Memory Allocation (Contd.)

- Let us see how this happens by allocating memory dynamically for the prime numbers.



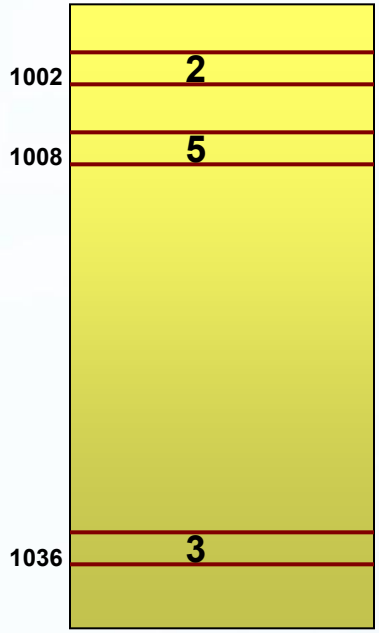
Memory allocated for 3

Memory representation

Data Structures and Algorithms

Dynamic Memory Allocation (Contd.)

- Let us see how this happens by allocating memory dynamically for the prime numbers.



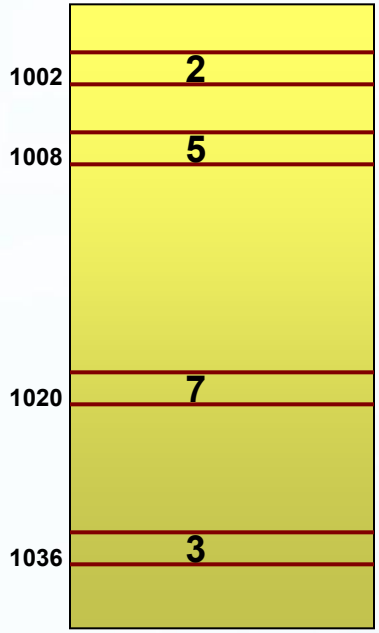
Memory allocated for 5

Memory representation

Data Structures and Algorithms

Dynamic Memory Allocation (Contd.)

- Let us see how this happens by allocating memory dynamically for the prime numbers.



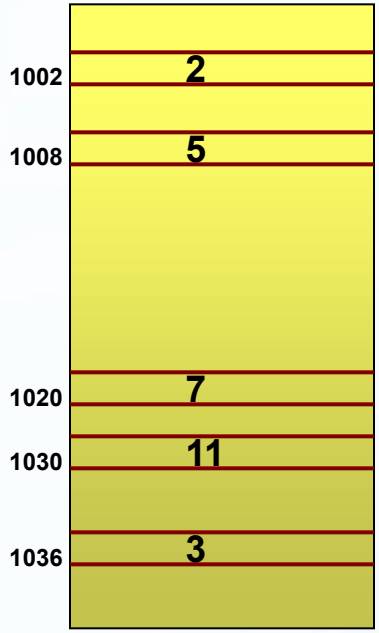
Memory allocated for 7

Memory representation

Data Structures and Algorithms

Dynamic Memory Allocation (Contd.)

- Let us see how this happens by allocating memory dynamically for the prime numbers.



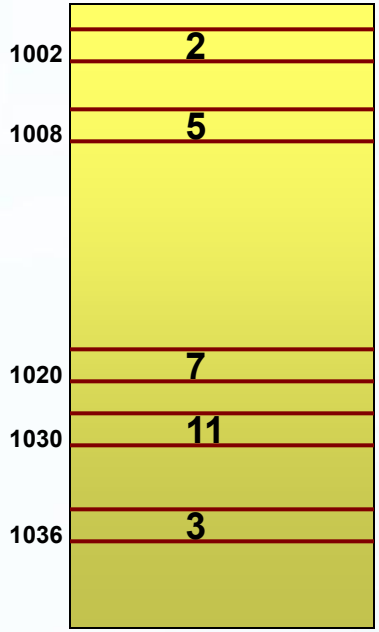
Memory allocated for 11

Memory representation

Data Structures and Algorithms

Dynamic Memory Allocation (Contd.)

- Now, if you know the address of the first element, you cannot calculate the address of the rest of the elements.
- This is because, all the elements are stored at random locations in the memory.
- To access any element, you need to know its address.

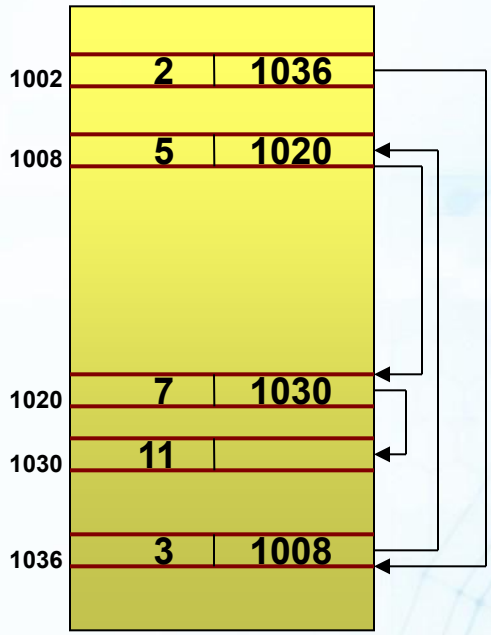


Memory representation

Data Structures and Algorithms

Dynamic Memory Allocation (Contd.)

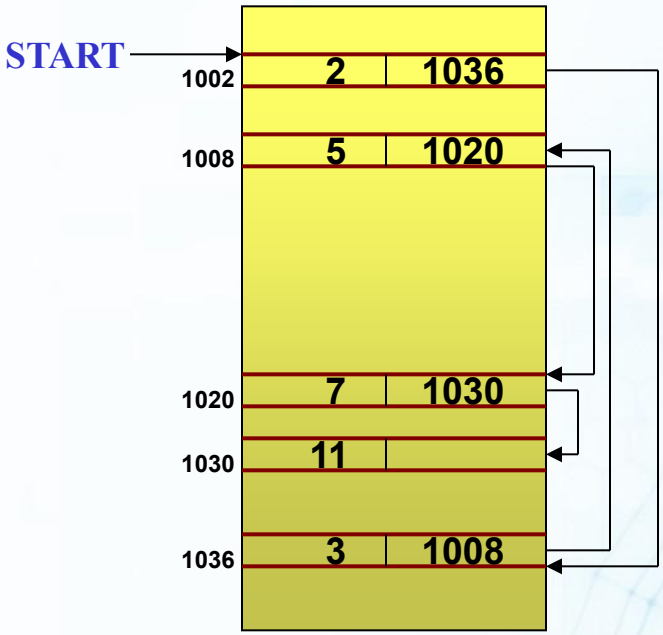
- Therefore, it would be good if every allocated block of memory contains the address of the next block in sequence.
- The following figure gives the list a linked structure where each block is linked to the next block in sequence.



Memory representation

Dynamic Memory Allocation (Contd.)

- You can declare a variable, `START`, that stores the address of the first block.
- The following figure displays that you can now begin at `START` and move through the list by using the links.

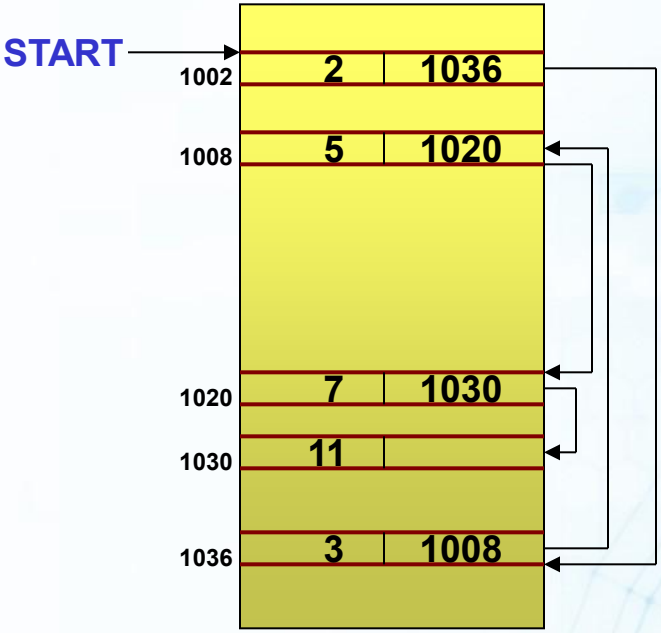


Memory representation

Data Structures and Algorithms

Dynamic Memory Allocation (Contd.)

- The following figure displays an example of a linked list, which is a data structure that implements the concept.



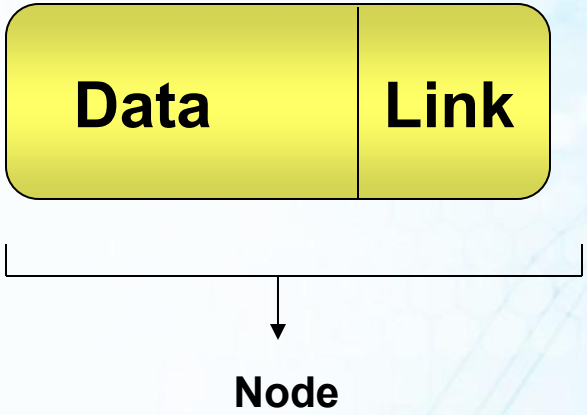
Memory representation

Defining Linked Lists

- Linked List:
  - Is a dynamic data structure.
  - Allows memory to be allocated as and when it is required.
  - Consists of a chain of elements, in which each element is referred to as a node.

Defining Linked Lists (Contd.)

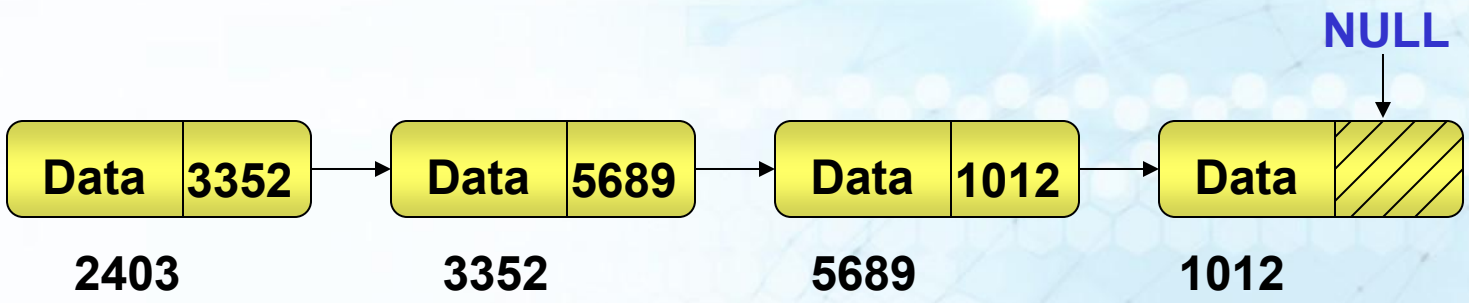
- A node is the basic building block of a linked list.
- A node consists of two parts:
  - Data: Refers to the information held by the node.
  - Link: Holds the address of the next node in the list.



Data Structures and Algorithms

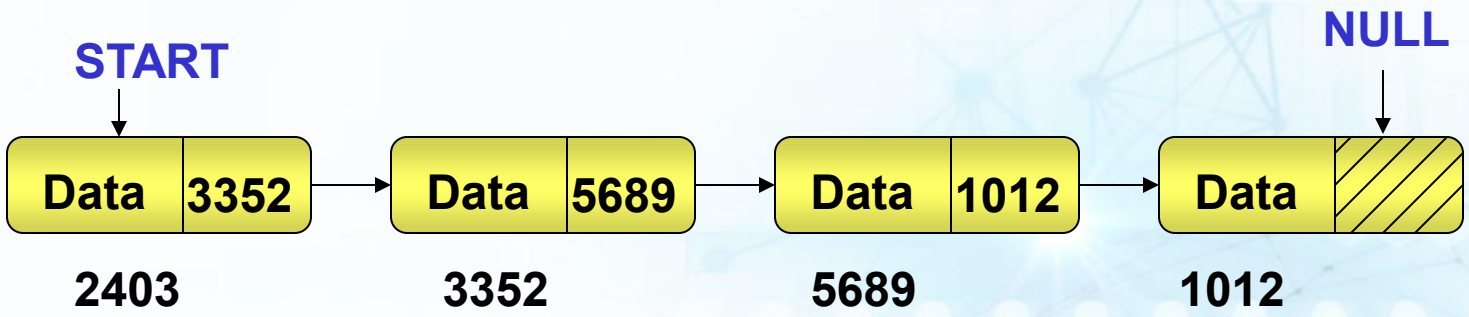
Defining Linked Lists (Contd.)

- All the nodes in a linked list are present at arbitrary memory locations.
- Therefore, every node in a linked list has a link field that stores the address of the next node in sequence.
- The last node in a linked list does not point to any other node. Therefore, it points to `NULL`.



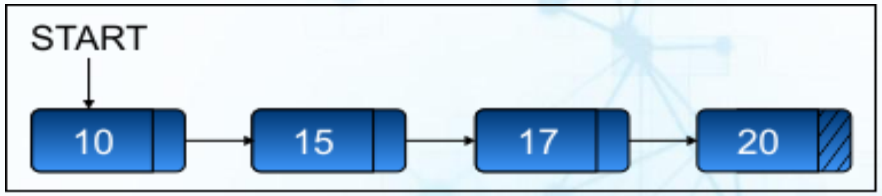
Defining Linked Lists (Contd.)

- To keep track of the first node, declare a variable/pointer, `START`, which always points to the first node.
- When the list is empty, `START` contains, `NULL`.

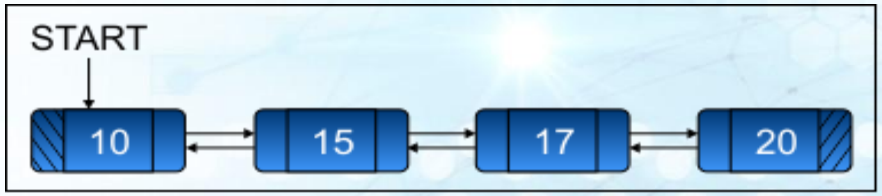


Identifying Different Types of Linked Lists

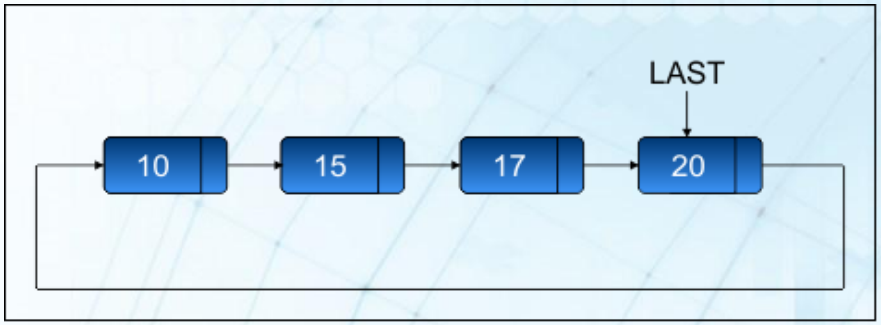
- Linked lists can be of the following types:
  - Singly-linked list



  - Doubly-linked list



  - Circular-linked list



Just a Minute

- Which one of the following linked lists has neither any beginning nor any end?
  - Singly-linked list
  - Doubly-linked list
  - Circular-linked list

Just a Minute (Contd.)

- Solution:
  - Circular-linked list

Implementing Singly-Linked Lists

- Let us solve the given problem of storing prime numbers using a linked list:

```
1. Repeat step 2 varying n from 2 to 1000000.
2. If n is a prime number, insert it in the
   linked list:
     a. Allocate memory for a new node.
     b. Store the prime number in the new node.
     c. Attach the new node in the linked list.
3. Display the prime numbers stored in the linked list.
```

Representing a Singly-Linked List

- A singly-linked list is represented in a program by defining two classes:
  - `Node` class: This class contains the data members which represent the data to be stored and the reference of the next node in sequence.

```
// Code in Java:
class Node
{
   public int info;
   public Node next; // Variable containing
                     // the address of the
                     // next node in sequence
 }
```

Data Structures and Algorithms

Representing a Singly-Linked List (Contd.)

```
// Code in C++:
class Node
 {
          public:
          int info;
          Node *next;    // Pointer to the
                         // next node in the sequence
  };
```

Data Structures and Algorithms

Representing a Singly-Linked List (Contd.)

- `List` class:
  - Consists of a set of operations implemented on a linked list, such as insertion, deletion, search, and traversal.
  - Contains the declaration of a variable/pointer, `START` that always points to the first node in the list.

```
// Code in Java:
class List
{
    private Node START;
    List()
    {
    START = NULL;
    }
```

Data Structures and Algorithms

Representing a Singly-Linked List (Contd.)

```
public void addNode(int element){}
public bool search(int element) {}
public bool delNode(int element) {}
public void traverse() {}
}
```

Data Structures and Algorithms

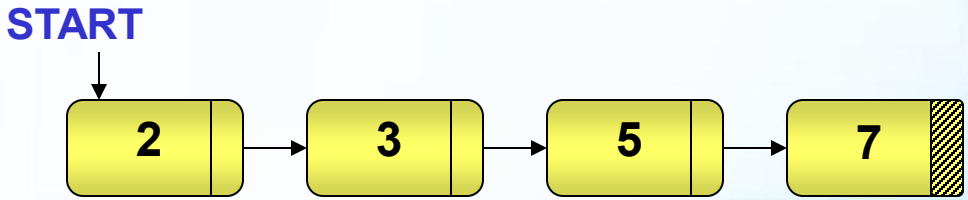
Representing a Singly-Linked List (Contd.)

```
// Code in C++:
class List
 {
  Node * START;
  public:
  List()
 {
 START = NULL;
 }
 void addNode(int element);
 bool search(int element, Node *previous,
 Node *current);
 bool delNode(int element);
 void traverse();
 };
```

Data Structures and Algorithms

Traversing a Singly-Linked List

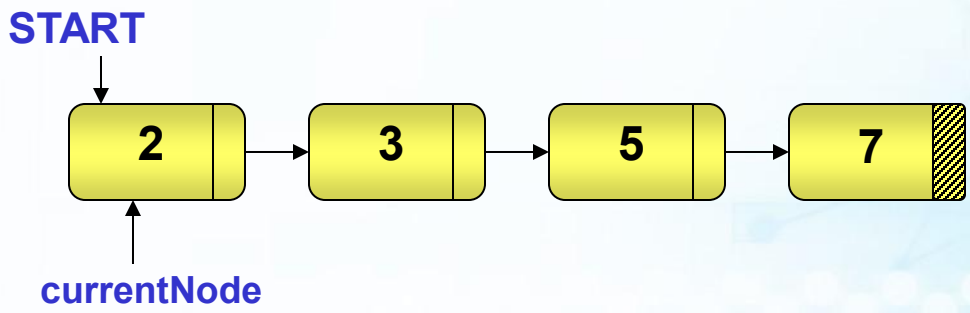
- Algorithm for traversing a singly-linked list:



1. Make currentNode point to the first node in the list.
2. Repeat the steps 3 and 4, until currentNode becomes NULL.
3. Display the information contained in the node marked as currentNode.
4. Make currentNode point to the next node in the sequence.

Data Structures and Algorithms

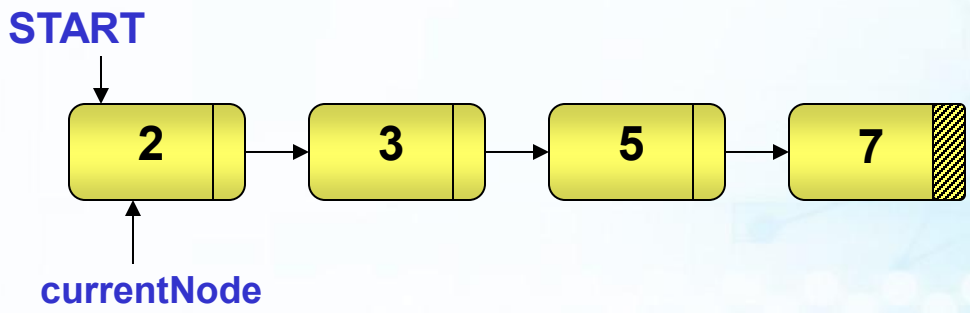
Traversing a Singly-Linked List (Contd.)



1. Make currentNode point to the first node in the list.
2. Repeat the steps, 3 and 4, until currentNode becomes NULL.
3. Display the information contained in the node marked as currentNode.
4. Make currentNode point to the next node in the sequence.

Data Structures and Algorithms

Traversing a Singly-Linked List (Contd.)



1. Make currentNode point to the first node in the list.

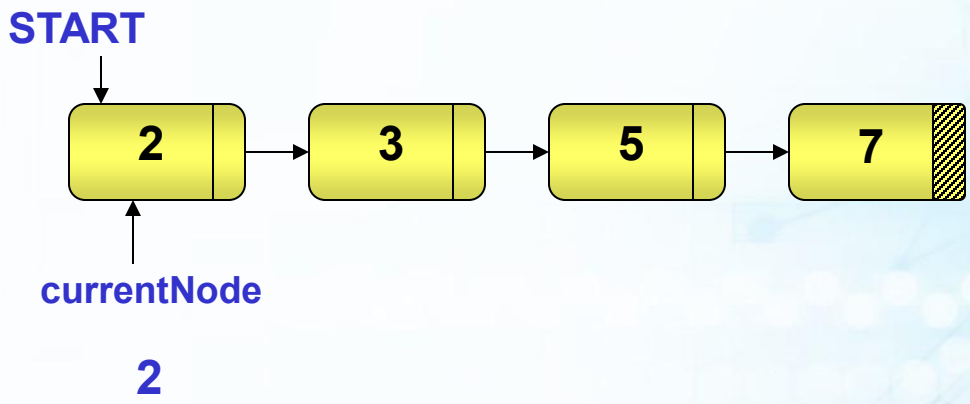
2. Repeat the steps, 3 and 4, until currentNode becomes NULL.

3. Display the information contained in the node marked as currentNode.

4. Make currentNode point to the next node in the sequence.

Data Structures and Algorithms

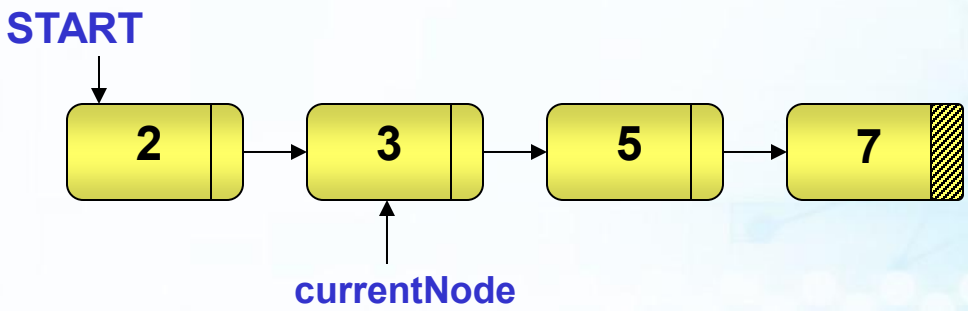
Traversing a Singly-Linked List (Contd.)



1. Make currentNode point to the first node in the list.
2. Repeat the steps, 3 and 4, until currentNode becomes NULL.
3. Display the information contained in the node marked as currentNode.
4. Make currentNode point to the next node in the sequence.

Data Structures and Algorithms

Traversing a Singly-Linked List (Contd.)

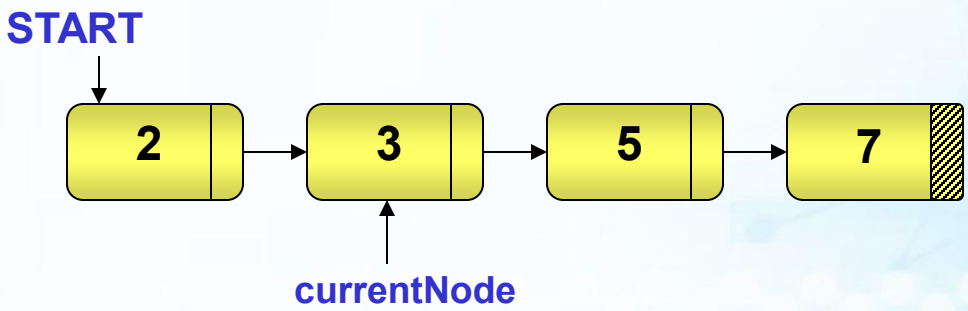


2

1. Make currentNode point to the first node in the list.
2. Repeat the steps, 3 and 4, until currentNode becomes NULL.
3. Display the information contained in the node marked as currentNode.
4. Make currentNode point to the next node in the sequence.

Data Structures and Algorithms

Traversing a Singly-Linked List (Contd.)

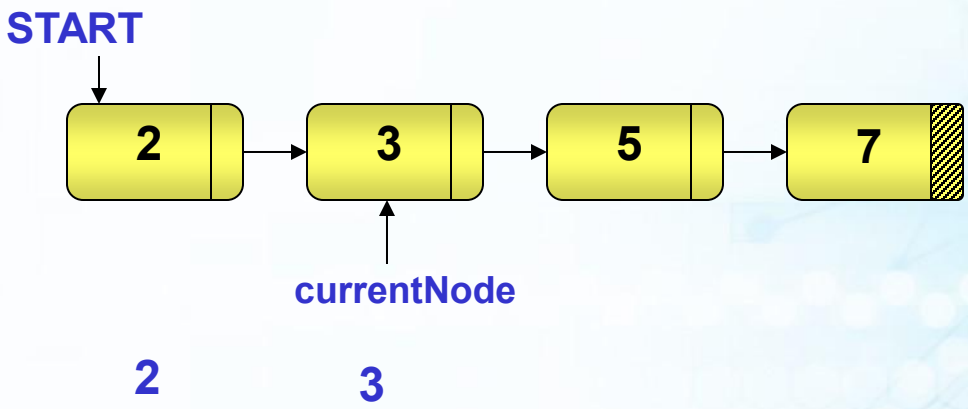


2

1. Make currentNode point to the first node in the list.
2. Repeat the steps, 3 and 4, until currentNode becomes NULL.
3. Display the information contained in the node marked as currentNode.
4. Make currentNode point to the next node in the sequence.

Data Structures and Algorithms

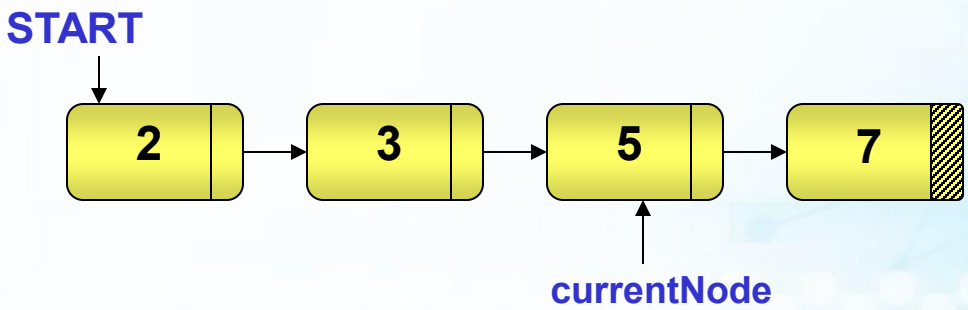
Traversing a Singly-Linked List (Contd.)



1. Make currentNode point to the first node in the list.
2. Repeat the steps, 3 and 4, until currentNode becomes NULL.
3. Display the information contained in the node marked as currentNode.
4. Make currentNode point to the next node in the sequence.

Data Structures and Algorithms

Traversing a Singly-Linked List (Contd.)

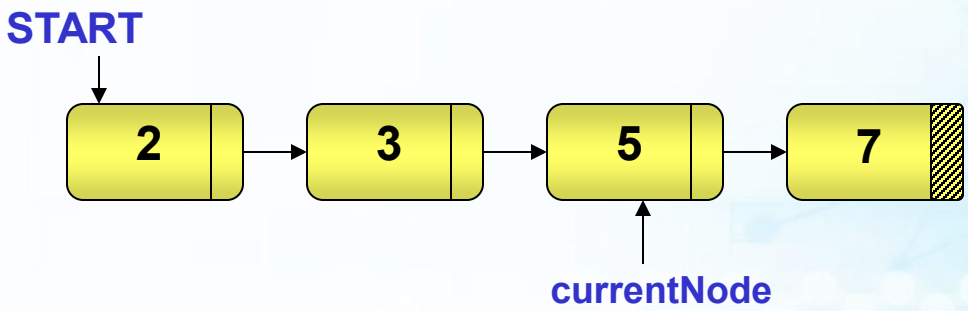


2 3

1. Make currentNode point to the first node in the list.
2. Repeat the steps, 3 and 4, until currentNode becomes NULL.
3. Display the information contained in the node marked as currentNode.
4. Make currentNode point to the next node in the sequence.

Data Structures and Algorithms

Traversing a Singly-Linked List (Contd.)

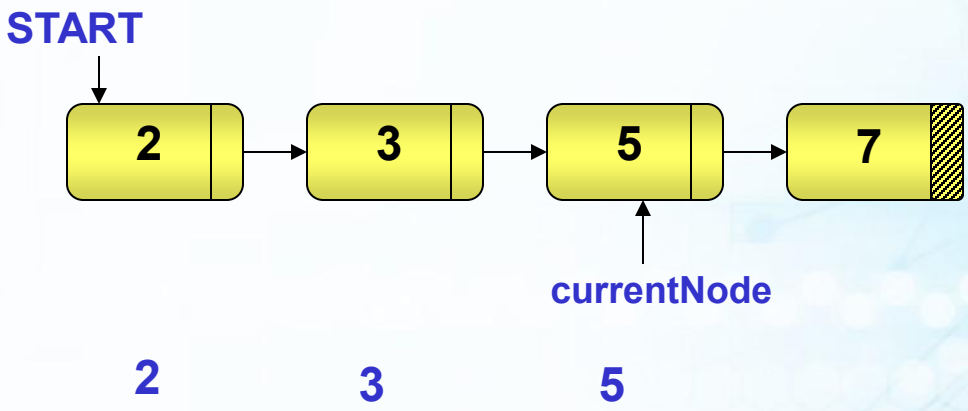


2 3

1. Make currentNode point to the first node in the list.
2. Repeat the steps, 3 and 4, until currentNode becomes NULL.
3. Display the information contained in the node marked as currentNode.
4. Make currentNode point to the next node in the sequence.

Data Structures and Algorithms

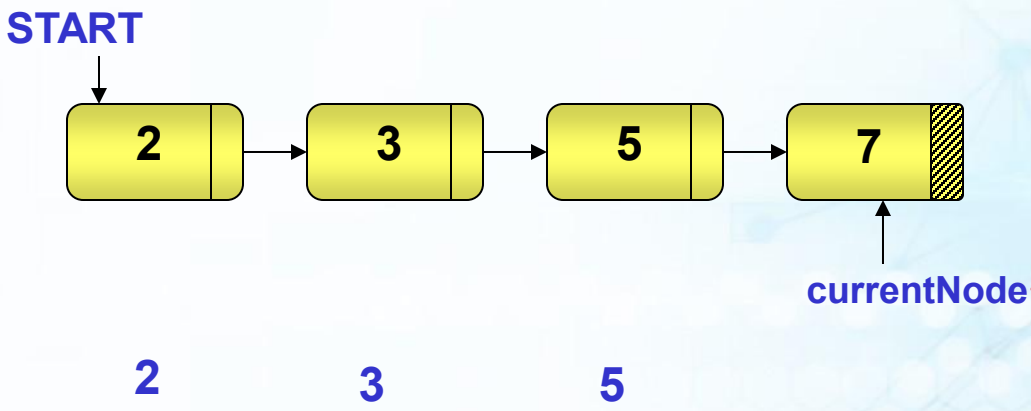
Traversing a Singly-Linked List (Contd.)



1. Make currentNode point to the first node in the list.
2. Repeat the steps, 3 and 4, until currentNode becomes NULL.
3. Display the information contained in the node marked as currentNode.
4. Make currentNode point to the next node in the sequence.

Data Structures and Algorithms

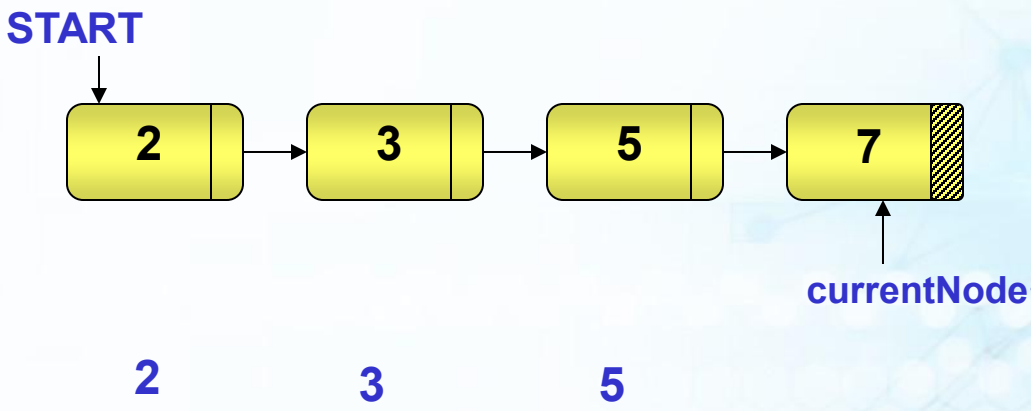
Traversing a Singly-Linked List (Contd.)



1. Make currentNode point to the first node in the list.
2. Repeat the steps, 3 and 4, until currentNode becomes NULL.
3. Display the information contained in the node marked as currentNode.
4. Make currentNode point to the next node in the sequence.

Data Structures and Algorithms

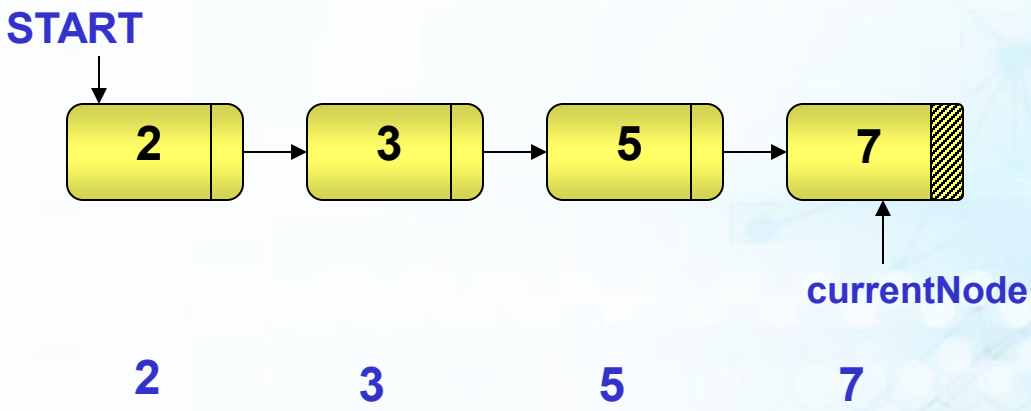
Traversing a Singly-Linked List (Contd.)



1. Make currentNode point to the first node in the list.
2. Repeat the steps, 3 and 4, until currentNode becomes NULL.
3. Display the information contained in the node marked as currentNode.
4. Make currentNode point to the next node in the sequence.

Data Structures and Algorithms

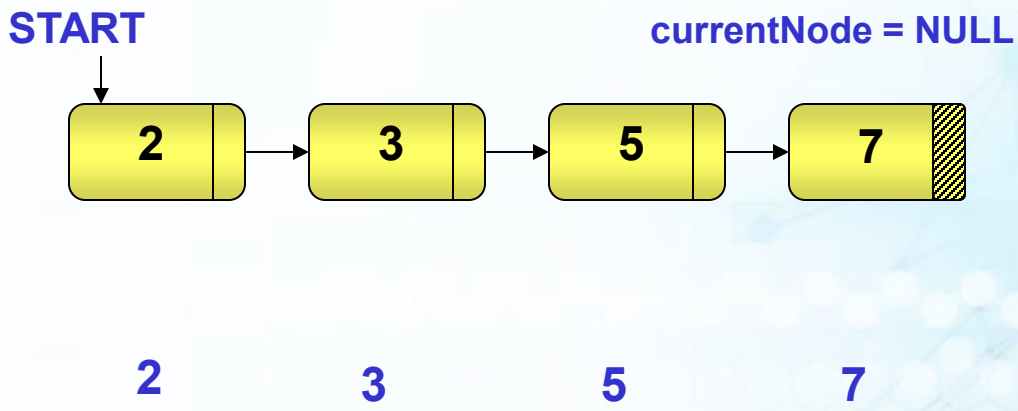
Traversing a Singly-Linked List (Contd.)



1. Make currentNode point to the first node in the list.
2. Repeat the steps, 3 and 4, until currentNode becomes NULL.
3. Display the information contained in the node marked as currentNode.
4. Make currentNode point to the next node in the sequence.

Data Structures and Algorithms

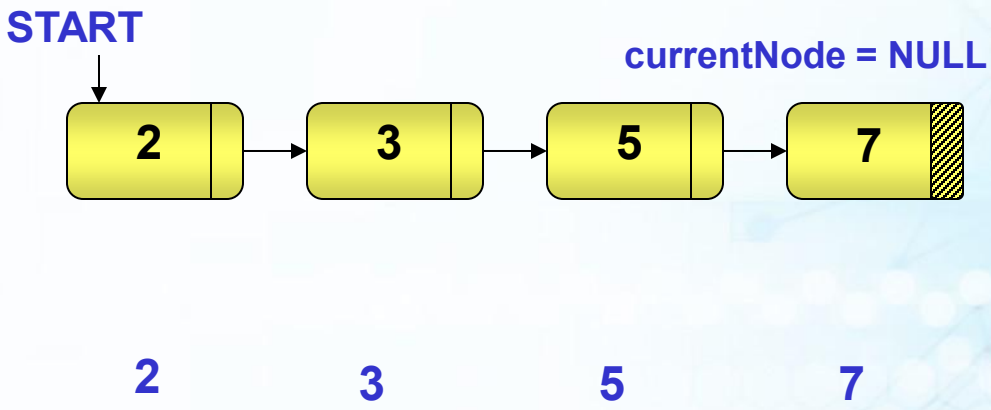
Traversing a Singly-Linked List (Contd.)



1. Make currentNode point to the first node in the list.
2. Repeat the steps, 3 and 4, until currentNode becomes NULL.
3. Display the information contained in the node marked as currentNode.
4. Make currentNode point to the next node in the sequence.

Data Structures and Algorithms

Traversing a Singly-Linked List (Contd.)



1. Make currentNode point to the first node in the list.
2. Repeat the steps, 3 and 4, until currentNode becomes NULL.
3. Display the information contained in the node marked as currentNode.
4. Make currentNode point to the next node in the sequence.

Traversal complete

Inserting a Node in a Singly-Linked List

- Insertion refers to the process of adding a new node in the list.
- The following algorithm depicts the logic to insert a node in an empty linked list:

```
1. Allocate memory for the new node.
2. Assign a value to the data field of the new node.
3. Make the next field of the new node point to NULL.
4. Make START point to the new node.
```

- If the linked list is not empty, you can insert a node at:
  - Beginning of the list
  - Between two nodes in the list
  - End of the list

Inserting a Node in a Singly-Linked List (Contd.)

- The following algorithm depicts the logic to insert a node at the beginning of the linked list:

```
1. Allocate memory for the new node.
2. Assign a value to the data field of the new node.
3. Make the next field of the new node point to START
   (that is the first node in the list).
4. Make START point to the new node.
```

Inserting a Node in a Singly-Linked List (Contd.)

- The following algorithm depicts the logic to insert a node at the end of the linked list:

```
1. Allocate memory for the new node.
2. Assign a value to the data field of the new node.
3. If START is NULL (if the list is empty), then:
    a. Make START point to the new node.
    b. Make LAST point to the new node.
    c. Go to the step 6.
4. Make the next field of LAST point to the new node.
5. Mark the new node as LAST.
6. Make the next field of the new node point to NULL.
```

Inserting a Node in a Singly-Linked List (Contd.)

Let us view an animation to understand the insertion of a node in an ordered singly-linked list.

Deleting a Node from a Singly-Linked List

- Delete operation in a linked list refers to the process of removing a specified node from the list.
- You can delete a node from the following places in a linked list:
  - Beginning of the list
  - Between two nodes in the list
  - End of the list

Deleting a Node from a Singly-Linked List (Contd.)

- The following algorithm depicts the logic to delete a node from the beginning of a list:

```
1. Mark the first node in the list as current.
2. Make START point to the next node in the sequence.
3. Release the memory for the node marked as current.
```

Deleting a Node from a Singly-Linked List (Contd.)

Let us view an animation to understand the deletion of a node in a singly-linked list.

Just a Minute

- A linked list is a chain of elements in which each element consists of data, as well as a _____ to the next element.

Just a Minute (Contd.)

- Solution:
  - Link

Demo: Implementing a Singly-Linked List

- Problem Statement:
  - Write a program to implement the insert, search, delete, and traverse operations on a singly-linked list, which stores the records of the students in a class. Each record holds the following information:
    - Roll number of the student
    - Name of the student

Demo: Implementing a Singly-Linked List (Contd.)

- Solution:
  - To implement insert, search, delete, and traverse on a singly-linked list that stores the records of students in a class, you need to perform the following tasks:
    1. Write the code.
    2. Execute the code and verify the output.

Summary

- In this session, you learned that:
  - The fundamental principle of hashing is to convert a given key value to an offset address to retrieve a record.
  - In hashing, conversion of a key to an address is done by a relation (formula), which is known as a hashing function.
  - The situation in which the hash function generates the same hash value for two or more keys is called collision.
  - Some methods of resolving collision are chaining and open addressing.
  - Dynamic memory allocation refers to the process of allocating memory on the basis of the need at runtime.
  - A linked list is a chain of elements in which each element consists of data, as well as a link to the next element.

Summary (Contd.)

- Based on how the various nodes are connected to each other, linked lists can be of the following types:
  - Singly-linked list
  - Doubly-linked list
  - Circular-linked list
- A singly-linked list is a simple data structure that acts as a base for the various other data structures.
- A singly-linked list is represented in a program by defining the following classes:
  - A class that represents a node in a linked list
  - A class that represents a linked list